QA / SQA Complete Master Cheat Sheet

Mutation Testing • Advanced Coverage • Test Design • Agile QA • Performance • Security • AI in Testing • Tools • Interview Q&A;

Muhammad Faizan | FAST-NUCES CS | Software Testing CS3046 | 2026

1. MUTATION TESTING

What is Mutation Testing?

Mutation testing evaluates the QUALITY of your test suite — not just coverage. It introduces small artificial faults (mutations) into the source code and checks whether your tests detect them. If a test fails after mutation, the mutant is KILLED (good). If tests still pass, the mutant SURVIVES (bad — test suite is weak).

Key Terms

- Mutant — a modified version of the original program
- Mutation operator — the rule used to modify code
- Killed mutant — at least one test detects the change
- Survived mutant — no test catches the change
- Equivalent mutant — mutant behaves same as original (cannot be killed — not a real fault)
- Mutation score — killed / (total - equivalent) x 100%

Mutation Score Formula

Score = Killed Mutants / (Total Mutants - Equivalent Mutants) x 100

Higher score = stronger test suite. Target: 80%+

Common Mutation Operators

- AOR — Arithmetic Operator Replacement: + to -, * to /
- ROR — Relational Operator Replacement: > to >=, == to !=
- COR — Conditional Operator Replacement: && to ||
- SVR — Scalar Variable Replacement: x to y
- SDL — Statement Deletion: remove a statement
- ABS — Absolute Value Insertion: x to abs(x)
- UOI — Unary Operator Insertion: x to -x or !x
- LCR — Logical Connector Replacement

Tools

- PIT (PITest) — Java mutation testing tool
- Stryker — JS/TS mutation testing
- mutmut — Python mutation testing

Deep Learning Mutation Testing (Your Research Papers)

Tool	What it does	Key idea
MuNN	Mutation testing for neural networks	Applies operators like weight perturbation, neuron activation change to NN layers
DeepMutation	Source-level and model-level mutations for DNNs	Mutates training data OR trained model weights/structure
DeepMutation++	Extended DeepMutation with more operators	Adds layer removal, filter change, Gaussian noise injection
DeepCrime	Mutation testing for DL training programs	Focuses on TRAINING CODE bugs — hyperparams, loss functions, optimizers

Q: What is mutation testing?

A: It evaluates test suite quality by introducing artificial faults called mutants. If your tests catch the fault, the mutant is killed. Mutation score = killed/total x 100%. A high score means your tests are thorough.

Q: What is an equivalent mutant?

A: A mutant that behaves identically to the original program despite code change. It cannot be killed by any test — not a real fault. Equivalent mutants reduce mutation score denominator.

Q: What is the difference between mutation score and code coverage?

A: Code coverage measures which lines are EXECUTED by tests. Mutation score measures whether tests actually DETECT faults. You can have 100% coverage with weak assertions and a low mutation score.

2. ADVANCED COVERAGE — SUBSUMPTION HIERARCHY

Complete Subsumption Chain — Strongest to Weakest

STRONGEST	All-def-use-pairs coverage (data flow)
	All-uses coverage
	All-defs coverage
	Prime Path Coverage (structural — one ring to rule them all)
	Edge-pair Coverage
	Edge (Branch) Coverage
WEAKEST	Node (Statement) Coverage

Logic Coverage Subsumption

Criteria	Abbr	Subsumes
Combinatorial Coverage	CoC	Everything below
Restricted Active Clause Coverage	RACC	CACC
Correlated Active Clause Coverage	CACC	GACC
General Active Clause Coverage	GACC	PC, CC
Predicate Coverage	PC	—
Clause Coverage	CC	—
Inactive Clause Coverage variants	GICC/RICC	Each other

3. TEST DOUBLES — MOCKS, STUBS, FAKES, SPIES

Type	What it is	When to use	Example
Dummy	Passed but never used	Fill required parameters	null object passed to a constructor
Stub	Returns hardcoded responses	Replace slow/unavailable dependency	Stub DB returns fixed user object
Fake	Working implementation, simplified	In-memory DB for testing	H2 in-memory database instead of PostgreSQL
Mock	Verifies interactions/calls were made	Test that a method was called	Verify emailService.send() was called once
Spy	Real object that also records calls	Partial mocking, track real usage	Spy on real service to check call count

Dependency Injection — Why it matters for testing

When dependencies are injected (not created inside class), you can swap them for test doubles. This is WHY your course covers dependency inversion — it makes code testable.

Mockito Example (Java)

```
// Mock creation
UserRepo repo = mock(UserRepo.class);
// Stub behaviour
when(repo.findById(1)).thenReturn(user);
// Verify interaction
verify(repo, times(1)).findById(1);
```

Dependency Inversion Principle

- High-level modules should not depend on low-level modules
- Both should depend on abstractions (interfaces)
- This allows swapping real DB for a test fake

Test DB / Layered Architecture

- Use separate test database (H2 in-memory)
- Layered arch: Controller > Service > Repo
- Test each layer independently with doubles

4. PERFORMANCE & NON-FUNCTIONAL TESTING

Type	Definition	Metric measured
Load Testing	Test under expected normal user load	Response time, throughput, error rate
Stress Testing	Push beyond limits to find breaking point	Max users, failure mode, recovery time
Spike Testing	Sudden large increase in load	How system handles sudden traffic burst
Soak/Endurance	Run under normal load for extended time	Memory leaks, degradation over time
Volume Testing	Test with large amounts of data	DB performance, file handling speed
Scalability Testing	Can system scale up to handle more load	Performance at 2x, 5x, 10x users
Baseline Testing	Establish performance benchmark	Reference point for future comparisons

Key Performance Metrics

- Response Time — time from request to response
- Throughput — requests handled per second
- Error Rate — % of failed requests
- Latency — network delay component
- Concurrency — simultaneous users supported
- Apdex Score — user satisfaction index (0-1)

Performance Testing Tools

- JMeter — open source, Java, most popular
- Gatling — Scala-based, good reports
- Locust — Python-based, scriptable
- k6 — JS-based, modern, CI-friendly
- Artillery — Node.js, cloud-native
- BlazeMeter — cloud-based JMeter

5. SECURITY TESTING

Attack/Concept	What it is	How to test for it
SQL Injection	Malicious SQL in input field to manipulate DB	Input: ' OR 1=1 -- in login field
XSS (Cross-site scripting)	Inject malicious JS into web pages	Input: alert(1) in form fields
CSRF	Trick user into submitting unintended request	Check for CSRF tokens on state-changing forms
Auth bypass	Access protected resource without valid token	Remove/modify JWT token, test without auth header
Broken access control	User accesses another user's data	Test tenant accessing landlord-only endpoints
Sensitive data exposure	Data sent/stored unencrypted	Check HTTPS, check DB for plain-text passwords
Rate limiting	No limit on login attempts — brute force risk	Send 1000 login requests, check if blocked
IDOR	Insecure Direct Object Reference — /users/1 -> /users/2	Change ID in URL to access others' data

Security Testing Tools

- OWASP ZAP — free web app scanner
- Burp Suite — intercept/modify HTTP requests
- Nikto — web server scanner
- Nmap — network port scanner
- SQLMap — automated SQL injection tool

OWASP Top 10 (must know)

- 1. Broken Access Control
- 2. Cryptographic Failures
- 3. Injection (SQL, NoSQL, OS)
- 4. Insecure Design
- 5. Security Misconfiguration
- 6. Vulnerable Components
- 7. Auth & Session failures
- 8. Software Integrity failures
- 9. Logging failures
- 10. Server-side Request Forgery

6. AGILE QA & SCRUM FRAMEWORK

Scrum Roles

- Product Owner — owns backlog, sets priorities
- Scrum Master — facilitates process, removes blockers

QA in Agile — Key Differences from Waterfall

- Testing is continuous, not a phase at the end
- QA is part of the team, not a separate department

- Development Team — devs + QA together

Scrum Events

- Sprint Planning — what to build this sprint
- Daily Standup — what did I do, what will I do, blockers
- Sprint Review — demo to stakeholders
- Sprint Retrospective — what to improve as a team

Sprint

Fixed time-box (1-4 weeks). QA works within same sprint as dev. No 'test later' — test NOW.

- Acceptance criteria defined BEFORE development
- Automated regression runs every sprint
- Exploratory testing done within each sprint

Definition of Done (DoD)

- Code written and reviewed
- Unit tests passing
- Integration tests passing
- Acceptance criteria met
- No critical bugs open

- Deployed to staging environment

Shift-Left Testing

Shift-left means involving QA EARLIER in the process — at requirements and design stage, not just after coding. Bugs found early cost 10x less to fix than bugs found in production. QA reviews BRS/SRS, writes acceptance criteria, and defines test cases BEFORE dev starts coding.

7. TEST DESIGN TECHNIQUES

Technique	What it is	Best for
Decision Table Testing	Map all combinations of conditions to actions	Business rules with multiple conditions
State Transition Testing	Test all states and transitions in a state machine	Order systems, login flows, workflows
Use Case Testing	Test user-facing scenarios from use case diagrams	End-to-end functional flows
Error Guessing	Experience-based — guess likely error locations	Exploratory, experienced QA
Cause-Effect Graphing	Graph relationships between causes and effects	Complex conditional logic
Pairwise Testing	Cover all pairs of inputs, not all combinations	Reduces test cases while maintaining coverage
Orthogonal Array	Statistical method for selecting test combinations	Large input spaces, parameter combinations

8. AI IN TESTING (Modern QA Awareness)

How AI is used in Testing

- Test case generation from requirements automatically
- Self-healing tests — AI updates broken locators
- Visual testing — AI detects UI layout regressions
- Log analysis — AI finds patterns in test failures

- Predictive analytics — which areas likely have bugs
- NLP — convert user stories to test cases

Deep Learning Mutation Testing (Your Research)

- MuNN — mutation operators for neural network layers
- DeepMutation — source-level + model-level mutations for DNNs
- DeepMutation++ — adds layer removal, noise injection operators
- DeepCrime — mutates DL training code: hyperparams, loss fn, optimizers

Why it matters for SQA role

Modern QA engineers need awareness of AI/ML testing challenges. Traditional test coverage metrics do not apply to neural networks. Mutation testing fills this gap by evaluating test suite quality for ML models.

AI-Powered Test Tools

- Testim — AI self-healing test automation
- Applitools — AI visual testing
- Functionize — NLP test creation
- Mabl — intelligent test automation
- Copilot / ChatGPT — generate test code

9. SELENIUM WEBDRIVER — IN DEPTH

Selenium Components

- Selenium IDE — browser plugin, record & replay
- Selenium WebDriver — programmatic browser control
- Selenium Grid — parallel execution on multiple machines/browsers
- Selenium RC — legacy, deprecated

Locator Strategies (priority order)

- ID — fastest, most reliable
- Name — second choice
- CSS Selector — flexible, fast
- XPath — powerful but slow
- LinkText / PartialLinkText — for links only
- TagName / ClassName — least specific, avoid

Waits (most asked)

- Implicit Wait — global, applies to all findElement calls
- Explicit Wait — waits for specific condition on specific element
- Fluent Wait — explicit wait with polling interval and ignore exceptions
- Thread.sleep() — hard wait, NEVER use in production tests

WebDriver Exceptions

- NoSuchElementException — element not found
- StaleElementReferenceException — element no longer in DOM
- TimeoutException — wait condition not met in time
- ElementNotInteractableException — element exists but not clickable

XPath vs CSS Selector

	XPath	CSS Selector
Speed	Slower (traverses DOM tree)	Faster (native browser support)
Direction	Can traverse UP (parent axis)	Can only go DOWN
Syntax	//input[@id='user']	#user or input[id='user']
Text match	//button[text()='Login']	Not supported natively
Prefer	When CSS cannot do it	Default choice — faster

10. BUG REPORTING & DEFECT MANAGEMENT

Good Bug Report Must Have

- Title — short, specific, actionable
- Environment — OS, browser, version, device
- Steps to Reproduce — numbered, exact
- Expected Result — what should happen
- Actual Result — what actually happened
- Severity — Critical / High / Medium / Low
- Priority — High / Medium / Low
- Attachments — screenshot, video, logs
- Test data used — specific inputs

Severity Levels

- Critical — system crash, data loss, security breach
- High — major feature broken, no workaround
- Medium — feature broken, workaround exists
- Low — cosmetic, typo, minor UI issue

Bug Tracking Tools

- Jira — most widely used in industry
- Bugzilla — open source
- Trello — lightweight, kanban
- GitHub Issues — for dev teams
- Azure DevOps — Microsoft stack
- TestRail — dedicated test management

Bad vs Good Bug Title

Bad (vague)	Good (specific)
Login doesn't work	Login fails with 500 error when password contains special characters on Chrome 120
Page crashes	Dashboard page crashes on load when user has 0 properties — NullPointerException in console
Button broken	Submit button on Register form does nothing when email field is empty — no validation message shown

11. TEST METRICS — WHAT QA ENGINEERS MEASURE

Metric	Formula / Definition	What it tells you
Defect Density	Bugs found / Size of software (KLOC or function points)	How buggy is the code per unit size

Defect Removal Efficiency	Defects found before release / Total defects x 100	How effective is QA at catching bugs early
Test Coverage	Test cases executed / Total test cases x 100	How much of plan is executed
Code Coverage	Lines/branches executed by tests / Total x 100	How much code is exercised by tests
Mutation Score	Killed mutants / (Total - Equivalent) x 100	How strong is your test suite
Pass Rate	Tests passed / Total tests run x 100	Overall build health
Defect Leakage	Bugs found in production / Total bugs x 100	How many bugs escaped QA
Mean Time to Detect	Average time to find a bug after introduction	QA speed and efficiency
Test Execution Rate	Test cases run per day	Team velocity
Blocked Test Cases	Test cases that cannot run due to dependency	Process bottlenecks

12. REVIEWS & STATIC TESTING

Review Type	Formality	Who participates	Output
Informal Review	None	Author + 1 reviewer	Verbal feedback
Walkthrough	Low	Author leads, peers attend	Issues list, learning
Technical Review	Medium	Technical peers, no mgmt	Issues list, decisions
Inspection	High	Trained moderator, defined roles	Formal defect log, metrics

Static vs Dynamic Testing

- Static — examine code/docs WITHOUT executing
- Dynamic — execute the software and observe
- Static: code reviews, inspections, linting, static analysis
- Dynamic: unit tests, integration, E2E, performance

Inspection Roles

- Author — created the document/code
- Moderator — leads the inspection process
- Reader — reads out the content
- Recorder — logs all defects found
- Inspector — reviews and finds defects

Static Analysis Tools

- SonarQube — code quality and security analysis
- ESLint — JS/TS linting
- Checkstyle — Java code style
- PMD — Java static bug finder
- SpotBugs — Java bytecode analysis

Benefits of Static Testing

- Find bugs before code even runs
- Cheaper than dynamic — no test environment needed
- Catches design flaws early
- Improves code maintainability

13. TESTING IN CI/CD PIPELINE

Test Pyramid

Layer	Test Type	Speed	Cost	Count
Top	E2E / UI Tests	Slow	High	Few (10-20)
Middle	Integration Tests	Medium	Medium	More (50-100)
Bottom	Unit Tests	Fast	Low	Many (500+)

CI/CD Pipeline Testing Order

- 1. Static analysis (linting, SonarQube)
- 2. Unit tests — fast, run on every commit
- 3. Integration tests — run on every commit
- 4. E2E/UI tests — run on PR merge or nightly
- 5. Performance tests — run weekly or on release
- 6. Security scans — run on release
- 7. UAT — manual, before production

Key CI/CD Concepts for QA

- Pipeline stages: build > test > deploy
- Test gates — pipeline fails if tests fail
- Parallel execution — run suites simultaneously
- Test reporting — HTML/XML reports per build
- Flaky tests — tests that randomly pass/fail
- Test isolation — each test independent, no shared state
- Rollback — revert deployment if tests fail in production

14. COMPLETE QA TOOLS ECOSYSTEM

Category	Tools
Unit Testing (Java)	JUnit 5, TestNG, Mockito
Unit Testing (JS)	Jest, Mocha, Chai, Jasmine
E2E Web Automation	Cypress, Playwright, Selenium WebDriver
Mobile Automation	Appium, Espresso (Android), XCTest (iOS)
API Testing	Postman, RestAssured (Java), Supertest (JS), Karate
Performance Testing	JMeter, Gatling, k6, Locust, Artillery
Security Testing	OWASP ZAP, Burp Suite, Nikto, SQLMap
Static Analysis	SonarQube, ESLint, Checkstyle, PMD, SpotBugs
Mutation Testing	PITest (Java), Stryker (JS), mutmut (Python)
CI/CD	Jenkins, GitHub Actions, GitLab CI, CircleCI, Azure DevOps
Test Management	TestRail, Zephyr, Xray, qTest
Bug Tracking	Jira, Bugzilla, GitHub Issues, Azure DevOps
Code Coverage	JaCoCo (Java), Istanbul/nyc (JS), Coverage.py (Python)
Visual Testing	Applitools, Percy, BackstopJS
Accessibility Testing	Axe, WAVE, Lighthouse
Cross-browser Testing	BrowserStack, Sauce Labs, LambdaTest
Contract Testing	Pact
Load Testing (cloud)	BlazeMeter, Gatling Cloud

15. COMPLETE INTERVIEW Q&A; — ALL TOPICS

Q: Why do you want to work in QA/SQA?

A: I built full-stack projects during my degree and consistently found that unstructured testing creates fragile systems. My Software Testing course gave me formal methods — TDD, coverage criteria, mutation testing. I want to apply engineering discipline to quality, not just find bugs randomly.

Q: What is the difference between QA and QC?

A: QA is process-oriented — prevent defects through good processes, audits, standards. QC is product-oriented — find defects in the actual product through testing and inspection. QA is proactive, QC is reactive.

Q: What are the 7 principles of software testing?

A: 1. Testing shows presence of defects not absence. 2. Exhaustive testing impossible. 3. Early testing saves cost. 4. Defect clustering — 80/20 rule. 5. Pesticide paradox — same tests stop finding bugs. 6. Testing is context-dependent. 7. Absence of errors fallacy.

Q: What is statement coverage vs branch coverage?

A: Statement (node) coverage: every line executed at least once. Branch (edge) coverage: every true/false branch of every decision executed. Branch subsumes statement — if you have 100% branch coverage you automatically have 100% statement coverage.

Q: What is prime path coverage?

A: Maximal simple paths in the CFG — no repeated nodes, cannot be extended. Prime path coverage tours every prime path. It subsumes edge-pair, edge, and node coverage. Our course called it the one ring to rule them all.

Q: What is mutation testing and mutation score?

A: Mutation testing injects artificial faults into code. If tests detect the change, mutant is killed. Mutation score = killed / (total minus equivalent) x 100. Higher score means stronger test suite. Different from code coverage — you can have 100% coverage with weak assertions and low mutation score.

Q: What is equivalence partitioning?

A: Divide input domain into groups where all values behave the same. Test one value per partition. Reduces test cases while maintaining meaningful coverage.

Q: What is boundary value analysis?

A: Test at edges of partitions — just below minimum, at minimum, just above minimum, just below maximum, at maximum, just above maximum. Bugs concentrate at boundaries.

Q: What is a decision table?
A: Table mapping all combinations of conditions to their resulting actions. Used when business rules have multiple conditions. Ensures all condition combinations are tested.

Q: What is pairwise testing?
A: Testing every PAIR of parameter values at least once instead of all combinations. Reduces test cases dramatically while catching most defects since most bugs involve interactions between two parameters.

Q: What is the difference between a mock and a stub?
A: Stub returns hardcoded/pre-configured responses to calls — used to isolate dependencies. Mock verifies that specific interactions occurred — used to assert a method was called with correct parameters. Stubs replace state; mocks verify behaviour.

Q: What is dependency injection and why does it matter for testing?
A: Injecting dependencies through constructor or setter instead of creating them inside the class. This allows swapping real implementations for test doubles. Code that uses new() inside methods is hard to unit test.

Q: What is the difference between smoke and regression testing?
A: Smoke: quick check on new build — is it stable enough to test? Core flows only. Regression: full re-run of all test cases after any code change to ensure nothing broke. Regression is comprehensive; smoke is a gate check.

Q: What is shift-left testing?
A: Involving QA earlier in SDLC — at requirements and design stage, not just after coding. QA reviews BRS, defines acceptance criteria before development. Bugs caught earlier cost exponentially less to fix.

Q: What is a flaky test?
A: A test that sometimes passes and sometimes fails without any code change — usually due to timing issues, test order dependency, or shared state. Flaky tests erode trust in CI pipelines and must be fixed or quarantined.

Q: What is test isolation?
A: Each test must be independent — setup its own data, not rely on other tests running first, clean up after itself. Violated isolation causes test order dependency and flaky behaviour.

Q: What is SQL injection?
A: Attacker inserts malicious SQL into input fields to manipulate the database. Example: username field receives ' OR 1=1 -- which bypasses authentication. Prevention: parameterized queries, prepared statements, ORM usage.

Q: What is OWASP Top 10?
A: OWASP's list of the 10 most critical web security risks. Top items: Broken Access Control, Cryptographic Failures, Injection, Insecure Design, Security Misconfiguration. Every QA engineer should know these as they define what to test for security.

Q: What is defect leakage?
A: Bugs that escaped QA and reached production. Formula: production bugs / total bugs x 100. Lower is better. High leakage means test coverage is insufficient or wrong areas are being tested.

Q: What is defect removal efficiency (DRE)?
A: Percentage of defects caught before production. Formula: bugs found before release / total bugs x 100. Industry target is 85-90%+. Measures overall QA effectiveness.

Q: How is QA different in Agile vs Waterfall?
A: Waterfall: QA is a phase after development, separate team, test at the end. Agile: QA is continuous, part of the dev team, test within each sprint, automation is mandatory. Agile QA writes acceptance criteria before development starts.

Q: What is definition of done?
A: Team agreement on what 'done' means for a user story. Typically: code written, code reviewed, unit tests passing, integration tests passing, acceptance criteria met, no critical bugs open, deployed to staging. QA enforces this.

16. LAST 10 MINS BEFORE INTERVIEW — SAY THESE OUT LOUD

" Mutation testing evaluates test QUALITY not just coverage — killed mutants / total gives mutation score. "

" Prime path coverage subsumes edge-pair, edge, and node coverage — one ring to rule them all. "

" CACC is the industry-standard active clause coverage — major clause independently determines predicate. "

" Shift-left means QA reviews requirements BEFORE dev writes code — bugs cost 10x less to fix early. "

" Test pyramid: many unit tests, fewer integration, few E2E — fast feedback at the bottom. "

" Flaky tests destroy CI trust — every flaky test must be fixed or quarantined. "

" DRE — defect removal efficiency — percentage of bugs caught before production. Target 85%+. "

" Mock verifies interaction. Stub returns hardcoded response. Fake is a working simplified implementation. "

" SQL injection prevention: parameterized queries. XSS prevention: output encoding. CSRF: tokens. "

" My JUnit5 framework achieved 84%+ JaCoCo code coverage with parallel execution and CSV data-driven tests. "

" RentEase Cypress suite covers auth, property, payment flows using Page Object Model — maintainable at scale. "

" Jenkins pipeline: install, test, publish HTML report — automated on every commit. "